Ders 6 Genişletilmiş Notu: Problem Uzayının Modellenmesi: Kavramsal Sınıflar, Bağlantılar, Öz Nitelikler ve Kavramsal Sınıf Diyagramı

Ders 6 Genişletilmiş Notu: Problem Uzayının Modellenmesi: Kavramsal Sınıflar, Bağlantılar, Öz Nitelikler ve Kavramsal Sınıf Diyagramı

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Geçen Haftaların Hatırlatması
- 2. Problem Uzayının Modellenmesi Nedir?
- 3. UML: Tümleştirilmiş Modelleme Dili
- 4. Sonucu Baştan Görmek: Kavramsal Sınıf Diyagramı
- 5. Kavramsal Modelden Yazılım Modeline: Öğrenci ve Ders
- 6. Kavramsal Sınıfları Bulmanın Yolları
- 7. Market Senaryosunda İsimleri İşaretlemek
- 8. Gereksiz Aday Sınıfların Elenmesi
- 9. Bir Aday Gerçekten Sınıf mı?
- 10. Betimleme (Tanımlama) Sınıfı: Gerçek Dünyadan Bilinçli Uzaklaşma
- 11. Bağlantıların Belirlenmesi
- 12. Bağlantı Örnekleri: Mağaza ve Havayolu
- 13. Market Satış Sisteminin Kavramsal Sınıf Diyagramı
- 14. Öz Niteliklerin Belirlenmesi
- 15. Modelin Son Hâli ve Modeli Okuyanlar
- 16. Analizin Sonu, Tasarıma Geçiş ve SRS Dokümanı

1. Geçen Haftaların Hatırlatması

Hoca derse, gereksinim analizini iki alt safhaya ayırdığımızı hatırlatarak başladı. Birinci safha iş analizidir: İhtiyaçlar belirlenir ve dokümante edilir. Bunun için iki yol görmüştük.

- **Business requirement dokümanı:** Sistemin yerine getirmesi gereken beklentiler maddeler ya da iş listesi hâlinde açıklanır, dikkat edilecek noktalar belirtilir.
- **Kullanım senaryosu (use case) modellemesi:** Sistemdeki her sorumluluk, birbiriyle ilişkili sorumluluklar da ilişkili senaryolar hâlinde modellenir. Senaryolar, o sorumlulukta yer alan aktörlerle yani paydaşlarla ilişkilendirilir. Bu ilişkiler iletişimi kolaylaştırmak için çok basit bir grafikle gösterilir. Ayrıca her sorumluluk için ayrı bir senaryo yazılır.

Gereksinimleri liste ya da senaryo olarak tanımlayabilirsiniz. İkisini de business requirement dokümanının içinde barındırabilirsiniz. Bu bilgileri oluşturmak için kullanılan araştırma ve bilgi toplama yöntemlerine de bakmıştık.

Gereksinimler ortaya çıktıktan sonra, tasarıma geçmeden önce problemin daha teknik ve formal bir tanımını yapmak gerekir. Buna çözümleme demiştik. Geçen hafta bunun ilk yolu olan işlevsel çözümlemeyi gördük. Veri akış diyagramları temelinde şu sorulara cevap aradık:

- Sistemin gereksinimleri hangi işlem birimlerine ayrıştırılmalı?
- Bu işlem birimleri çalışmak için hangi varlıklarla ilişki kurmalı, yani kimden veri almalı, kime veri vermeli?
- Veri nerede depolanmalı, oralara hangi veriler akmalı, oralardan hangi veriler alınmalı?
- İşlemlerin iç yapıları, karar ve kontrol mekanizmaları nasıl olmalı ve nasıl ifade edilmeli?

Son sorunun cevabı süreç tanım formlarıydı. Bu formların içinde koşullar ve çalışma şekli sözde kod (pseudocode), karar ağacı ya da karar tablosuyla aktarılıyordu.

Bu haftanın başlığı, işlevsel çözümlemenin alternatifidir. Nesneye dayalı bir sistem geliştiriyorsam ya da nesneye yönelik bir modelleme yapmak istiyorsam, analizin ikinci aşaması olan çözümlemeyi nasıl yaparım? Yani nesneye yönelik çözümleme nasıl yapılır? Bu dersin sorusu budur.

2. Problem Uzayının Modellenmesi Nedir?

2.1 Neden Bu İsim?

Konuya "nesneye yönelik çözümleme" de diyebiliriz. Hoca yine de "problem uzayının modellenmesi" adını tercih ediyor. Çünkü bu bir modellemedir, tasarıma hazırlık aşamasıdır ve esas amaç kavramsal bir çözümleme yapmaktır. Bunun için önce problemin yapısını bir uzayda tanımlamak ve modellemek gerekir.

Amaç şudur: Gerçek dünyada yer alan ve çözmemiz gereken problemi yazılım dünyasına taşımak için **doğru, anlaşılabilir ve sınanabilir (test edilebilir) bir replika model** oluşturmak. Bu aşamayı çok daha basit bir şekilde "problemin anlaşılması aşaması" olarak da ifade edebiliriz. Başka bir okumayla bu, gerçek dünya kavramları üzerinden bir yazılım çözümü taslağı oluşturmaktır.

2.2 İşlevsel Çözümlemeden Farkı

İki yaklaşımın baktığı yer farklıdır.

- **İşlevsel çözümleme:** Sistemde yerine getirilmesi gereken işlemleri ve işlevleri bulur. Bunları birimler hâlinde bir araya getirir. Bu birimler birlikte çalışarak sorumlulukları yerine getirir.
- **Problem uzayının modellenmesi:** Problemde yer alan, ona destek veren ya da paydaşı olan bütün kavramları bulur. Bu kavramların yazılımda karşılıklarını oluşturur. Yani problemin yazılım olarak mümkün olduğunca yakın bir replikasını kurar.

Hoca bir de iki hafta önceki iş analiziyle karşılaştırma yaptı. İş analizi nesneye dayalı değildi. Use case yazsak bile Türkçe ya da İngilizce normal cümleler yazıyorduk. Nesneye dayalı çözümleme asıl bu aşamada başlar: Yazılım nesnelerini oluşturmak üzere aday kavramlarımızı, aday sınıflarımızı bulup ortaya çıkarırız. Bazı istisnalar olacaktır, ama modelde mümkün olduğunca gerçek dünyadaki kavramları kapsayan sınıflar ve onlardan türeyen nesneler bulunur.

2.3 Modelde Neler Olur, Neler Olmaz?

Modelde üç şey yer alır:

- **Nesneler:** Yazılımda işlev ve veri tutacak birimlerdir.
- **Özellikler:** Nesnelerin tuttuğu, kendilerine has özelliklerdir.
- **İlişkiler (bağlantılar):** Bir kavram başka bir kavramla ilişkidedir. Ona ya hizmet eder ya da ondan hizmet alır. Farklı kavramlar bu ilişkiler üzerinden etkileşime girer ve iş birliği içinde çalışarak bir sorumluluğu yerine getirir.

Problemi ve kavramları modellediğimiz için yazılım dünyasındaki ayrıntı nesneleri düşünmeyiz.

> **Hocanın vurgusu:** Kavramsal modelde yazılım ayrıntısı düşünülmez. Grafik arayüzde hangi kütüphane ve hangi sınıf kullanılacak, veri tabanına hangi araçla erişilecek, veri taşıma nesneleri neler olacak gibi sorular bu aşamanın konusu değildir.

Bizim için önemli olan problemdeki kavramlardır: Satış, stok gibi. Günlük hayatta işler hangi kavramların bir arada çalışmasıyla yapılıyorsa bunu yazılım uzayına birebir taşımaya çalışırız.

3. UML: Tümleştirilmiş Modelleme Dili

Çözümü yine basit diyagramlarla ve görsel olarak ifade ederiz. Bunun için özel bir dil kullanırız: **UML (Unified Modeling Language)**. Hoca ayrıntısına girmeyeceğini söyledi. UML bir diyagram ailesi, daha doğrusu bir modelleme dilidir.

"Tümleştirilmiş" denmesinin sebebi şudur: Gereksinimlerin ortaya konmasından sistemin çalışır bir ürün hâline gelmesine kadar, bütün safhalarda ifade edilmesi gereken her şey için grafik araçlar sunar ve bunları tek bir aile olarak verir.

- **Analizde** bir diyagramı belli bir şekilde kullanırsınız.
- **Tasarımda** aynı diyagramı değiştirerek kullanırsınız.
- **Gerçekleştirmede** bu diyagramlardan yola çıkarak kodlama yaparsınız.
- **Devrede tutarken** bile bileşenleri ve barındırmaları aynı ailedeki diyagramlarla modellersiniz.

Görsel modelleme iki fayda sağlar. Birincisi, oluşturulacak sistem paydaşlar tarafından daha rahat anlaşılır. İkincisi, görselleştirilmiş analiz modeli tasarıma girdi olur. Tasarımda sorumlulukları ilgililerine atayacağız: Bir işin yapılması için gerekli eylemler hangi kavramlara nasıl dağıtılsın? Bu soruyu cevaplamadan önce o kavramların oluşturulmuş olması gerekir. Bu hafta o kavramları oluşturuyoruz.

4. Sonucu Baştan Görmek: Kavramsal Sınıf Diyagramı

Hoca, önce sonucu gösterip sonra oraya nasıl gelindiğini anlatmayı seçti. Bunu, başında sonunu gördüğünüz, sonra olay örgüsünü geriye doğru öğrendiğiniz filmlere benzetti: "Bu yüzden böyle olmuş, bu şuradan geliyormuş" diyerek sonuca tekrar varırsınız.

Slayttaki diyagramda şunlar görülür:

- **Kutular (dörtgenler):** Her kutunun iki alanı vardır. Birinde kavramın adı, ötekinde bir liste yani kavramın o an görebildiğimiz öz nitelikleri yazar.
- **Çizgiler:** Kutuları birbirine bağlar. Demek ki bu kavramlar ilişkilidir. İlişkinin ne olduğu çizginin üzerine, burada İngilizce olarak yazılmıştır.
- **Çoğullamalar:** Çizginin uçlarında, o ilişkide her kavramdan kaç tane yer aldığı yazar. Örneğin bir uçtaki kavram tek bir tane, öbür uçtaki sıfır ya da en fazla bir tane yer alıyor olabilir.

Bu resim bir **sınıf diyagramıdır (class diagram)**. Bu aşamada kavramsal olduğu için **kavramsal sınıf diyagramı (conceptual class diagram)** denir ve analiz modelini ifade eder.

Yaptığımız işin özü şudur: Çözülecek problem için yazılım temelli bir sistem geliştiriyoruz. Problem uzayı ile yazılım uzayının mümkün olduğunca özdeş olmasına, hatta aynı kavramları kullanmasına gayret ediyoruz. Tasarımda sorumlulukları atarken de bu modeli kullanıyoruz.

5. Kavramsal Modelden Yazılım Modeline: Öğrenci ve Ders

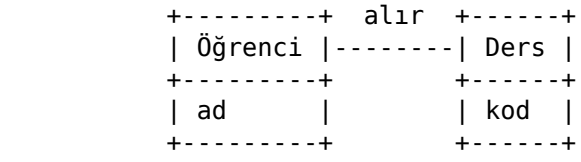
Hoca bu dönüşümü bir öğrenci bilgi sistemiyle gösterdi. Gereksinimlerden ya da kullanım senaryolarından iki kavram bulduğumuzu varsayalım: öğrenci ve ders. Dersin onu tanımlayan bir kod numarası, öğrencinin de adı vardır. Bir öğrenci hiç ders almayabilir ya da istediği kadar, yani n tane ders alabilir.

Soru: Öğrencinin numarası yok mu? **Cevap:** Hocanın kendi sorup cevapladığı bu soruda cevap şudur: Var, ama bu modelde yok. Örnekte özel bir kursun öğrencileri yalnızca isimleriyle tuttuğunu, numaralandırma yapmadığını varsayıyoruz.

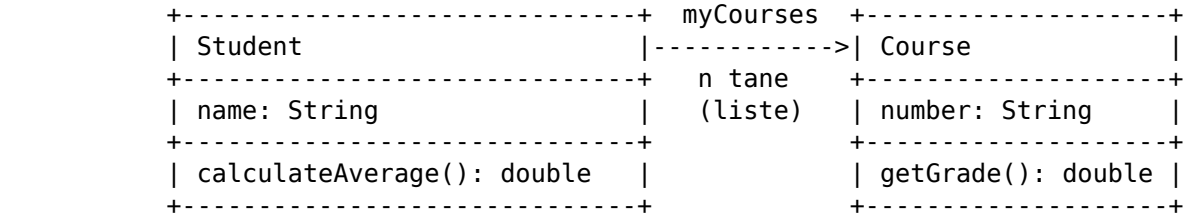
Bu modele bakınca şunu görürüm: Sistemde öğrenci ve ders diye iki kavram vardır, aralarında belli sayılarla kurulan belli bir ilişki vardır.

Tasarıma geçince yine dikdörtgenler görürüz, ama artık bunlar yazılım sınıflarıdır:

Kavramsal model



Yazılım modeli (tasarım)



Yazılım sınıflarını hoca şöyle okudu:

- `Student` sınıfının string tipinde `name` öz niteliği vardır.
- Öğrencinin ortalamasını hesaplamak için bir davranışı, yani metodu vardır. Bu metot gerçek sayı (`double`) döndürür.
- İki sınıf arasında yönlü bir ilişki vardır: Bir öğrencinin, `myCourses` isimli liste yapısında duran n tane `Course` nesnesi vardır.
- `Course` nesneleri `Course` yazılım sınıfından türer. Bu sınıfın string tipinde bir numarası vardır.
- Her kursun `getGrade` metodu vardır. Kurs, sorulduğunda notu `double` olarak söylemekle yükümlüdür.

Yazılımda da birebir `public class Student` ve `public class Course` yazacağım ve `Student` içinde bir `Course` listesi tutulması gerektiğini anlayacağım. Bakın, kavramsal modelde öğrenci ve ders varken yazılımda da öğrenci ve ders var. Kavramsal sınıf diyagramı, yani analiz modeli, bu dönüşüme temel sağladığı için önemlidir. Çözümü onu ele alarak oluşturmaya başlarız.

6. Kavramsal Sınıfları Bulmanın Yolları

Problemde yer alan bütün parçaları nasıl ortaya çıkaracağız? Buna kavramsal sınıfların bulunması denir. Problemin gerçek dünyasında yer alan somut ya da soyut varlıkları bulup sınıf olarak modellemeye çalışırız. Bazı kavramlar soyut olabilir. Hoca üç yol anlattı.

6.1 Kategori Listesi Kullanmak

Dünyada sıfırdan bilgi sistemi yapan ilk kişi değilim. Daha önce binlerce nesneye yönelik bilgi sistemi yapıldı. Bir kısmını ben ya da ekibim yapmış olabilirim, ya da benzer iş yapanlara sorup öğrenebilirim. O hâlde deneyimle, bu tür problemde sık kullanılan kavramların neler olduğu bilinir.

Hocanın örneği öğrenci bilgi sistemidir. Akla şunlar gelir: öğrenci, ders, bölüm, fakülte, ders tipleri (zorunlu ders, seçimlik ders), staj, transkript, eğitim planı.

Bu yolun bir riski vardır. Belli bir probleme özel sistem geliştiriyorsanız genel listeden seçim yapmak, özel kavramları gözden kaçırmanıza sebep olabilir. Bu yüzden çok beylik projelerde kullanılabilir, özel işlerde çok mantıklı bir yaklaşım değildir.

6.2 Yayınlanmış Modellerin Uyarlanması

Var olan bir modeli güncellemektir. Bunun için elinizde nesneye yönelik bir kavramsal model olması gerekir. Örneğin projenin çalışan bir sürümü ve onun analiz ve tasarım modeli vardır. Oradaki modelden başlar, yalnızca gerekli sınıf ilavelerini ve güncellemeleri yaparsınız. Ama bu her zaman mümkün değildir. Problemi ilk defa çözüyor olabilirsiniz.

6.3 Use Case'lerdeki İsimleri ve İsim Tamlamalarını Kullanmak

Hem özel hem de ilk defa karşılaşılan işlere, genel olarak bütün projelere uygulanabilecek bir yol var mı? Evet: İş analizinde yazdığımız use case'leri kullanmak. Senaryolardaki **isimleri ve isim tamlamalarını** alırız. Hoca bunun biraz Türkçe dersine benzeyeceğini söyledi.

> **Hocanın vurgusu:** Use case'lerdeki bütün isimler ve isim tamlamaları problem uzayınızda yer alabilecek adaylardır. Bunlara aday sınıf, aday kavram denir. Ancak hepsini kavram olarak modellerseniz gereksiz, tekrar eden ya da bir kavramı daha özel veya daha genel açıklayan ifadeler görürsünüz. Bu yüzden eleme şarttır.

Beklentinin ötesinde çok sayıda sınıf çıkabilir. Gerçekten problemi oluşturan parçaları tutarız. Kalanı ya ilgili kavramın içine ekler ya da "sınırlarımızın dışında" veya "çözümün bu iterasyonunu ilgilendirmiyor" diyerek dışarıda bırakırız.

7. Market Senaryosunda İsimleri İşaretlemek

Hoca bir use case'in ana başarılı senaryosunu gösterdi. Use case'in tepesinde kod ve ad, başlık alanı, birincil ve destek aktörler, ilgililer ve beklentileri, ön koşullar ve son koşullar vardı. Bunların altında işin nasıl yapıldığını anlatan akış yer alıyordu. Akış 10 adımdan, yani birbirini takip eden 10 cümleden oluşuyordu.

Hoca isimleri fosforlu kalemle işaretler gibi tek tek gösterdi. Anlatımda açıkça geçen adımlar şunlardır:

- **Adım 1:** Müşteri (customer), satın almak üzere ürün ve hizmetlerle (goods, services) kasaya (POS checkout) gelir. "Purchase" fiildir, işaretlenmez.
- **Adım 2:** Kasiyer (cashier) yeni bir satış (new sale) başlatır.
- **Adım 3:** Kasiyer ürün kimliğini (item identifier), örneğin barkod numarasını girer.
- **Adım 4:** Sistem satış satırını (sale line item) kaydeder. Ekranda ürün tanımını (item description), fiyatı (price) ve toplam tutarı (total) gösterir. Fiyat, fiyat kurallarıyla (price rules) oluşturulur.
- **Sonraki adımlar:** Hoca burada numara vermeden vergileri (taxes), ödemeyi (payment), envanteri (inventory), komisyonu (commission), muhasebeyi (accounting) ve fişi (receipt) işaretledi.
- **Adım 7:** Müşteri ödemeyi yapar, sistem ödemeyi işler.
- **Alternatif akış 7a:** Nakit ödemede verilen miktar girilir. Sistem kalan tutarı gösterir: Ya verilmesi gereken para üstünü ya da daha fazla ödenmesi gereken tutarı.

Fiyat kurallarına hoca şu örneği verdi: Bir ürünün normal fiyatı 10 lira olabilir, ama müşteri kartıyla gidince indirim yapılabilir.

7.1 Sayısal Örnek: Ekrandaki Toplam

Hoca dördüncü adımı somutlaştırdı.

- **Verilen:** Ürün kimliği "869…" ile başlayan bir barkod, ürün tanımı "çikolatalı gofret", birim fiyat 10 TL, adet 3.
- **İşlem:** $3 \times 10 = 30$.
- **Sonuç:** Toplam 30 lira.

Daha önce başka ürünler de alındıysa ekranda o ana kadarki toplam tutar yazar.

7.2 İlk Eleme Düşüncesi

Bütün isimleri işaretledik, ama dördüncü satıra bakınca bile bir şey fark ediyoruz. Satış kalemi, bir şeyin içinde yer alacak bir satıra benziyor. Ürünün tanımı, kod numarası, fiyatı ise kavramdan çok özellik gibi görünüyor. Sürekli "ürünün bir şeyi" diyoruz. Yukarıda "good" dedik, burada "item" diyoruz. Galiba bunlar aynı kavram, bazıları da o kavramın özellikleridir.

Bir ürünün kod numarası tek başına bir kavram mıdır? Hocaya göre değildir, ürüne has bir özelliktir. Demek ki hepsine aday sınıf dersek kocaman ve gereksiz ayrıntılarla dolu bir aday sınıf uzayı çıkar. Bu yüzden eleme yapılır.

8. Gereksiz Aday Sınıfların Elenmesi

Hoca yedi tür gereksiz aday saydı.

8.1 Fazlalık (Redundant) Sınıflar

Aynı kavramı ifade eden birden fazla kelimedir. Bir yerde good, bir yerde item, bir yerde product yazıyor olabilir. Hepsi tek bir varlığı anlatır: ürün.

Bir örnek de kişi ve müşteridir. Kişi daha genel, müşteri daha tanımlayıcı bir ifadedir. Kişiyi eleyip müşteriyi tutabilirsiniz. Bu noktada senaryoları da güncellersiniz: "Burada kişi demişiz, aslında bu müşteri." Bazen de "kişi" dediğiniz şeyin kasiyer ya da müdür olduğunu görürsünüz. Böylece kavramlar netleşir. Analiz modeli ve iş analizi iteratif olarak birlikte güncellenir.

8.2 İlgisiz Sınıflar

Bir senaryo için sınıf diyagramı oluştururken, çözümün o aşamasıyla ilgisi olmayan bir sınıf görebilirsiniz. Satışın yapılması, ürünlerin okutulması, tutarın ve vergilerin hesaplanması ve ödemeye aktarılma akışına odaklanmışken ödemenin içinde "kredi kartı" lafı geçmiş olsun. Kredi kartı elbette sistemin bir kavramıdır ve ödeme senaryosunda çok daha fazla geçecektir. Son diyagramda da yer alacaktır. Ama bu senaryo için gerekli olmadığından şu anda modelin dışında tutulabilir.

8.3 Belirsiz Sınıflar

Sınırları iyi çizilmemiş, geniş ve muğlak ifadelerdir. Genellikle ya birden fazla sınıftan oluşan bileşik bir kavramdır ya da başka bir sınıfın alt parçasıdır. Bu sorun senaryoların düzgün yazılmamasından doğar. Senaryolar düzgün yazılsaydı böyle ifadeler olmazdı.

Örnek: "Bilgileri muhasebe sistemine kaydedilir." Muhasebe sistemi kendi başına ayrı bir dünyadır, kim bilir kaç sınıftan oluşuyor. Aslında bahsettiğimiz, onun bir arayüzü ya da eriştiğimiz bir temas noktasıdır. Bu yüzden "muhasebe sisteminin şu arayüzü" ya da "şu erişim noktası" diye net yazmak daha doğrudur.

8.4 Nitelikler (Özellikler)

Kendi başına kavram olarak var olması anlamlı olmayanlardır. Barkod numarası ve birim fiyat, ürün sınıfının özelliğidir. Kendi başlarına bir şey ifade etmezler. Elenir ve ait oldukları sınıfa aktarılırlar.

8.5 İşlemler

Yanlış işaretlemeyle aday listesine fiiller ya da sorumluluklar da girmiş olabilir. Bunlar tek başına sınıf olmamalıdır. Çünkü bunlar etkileşimdir ve kavramların davranışları olarak modellenmelidir. Hocanın örneği ödemedir: Ödemeyi kabul etmek, toplam tutarı hesaplamak ya da para üstünü hesaplamak ayrı kavramlar değil, davranışlar ya da işlemlerdir.

> **Ders sonrası not:** Örnek, "ödeme" kavramının hiç sınıf olmayacağı anlamına gelmez. Bölüm 12 ve 13'teki diyagramlarda `Payment` bir sınıftır. Ayrım şudur: Varlık olarak ödeme bir kavramsal sınıftır. "Ödemeyi kabul et", "para üstünü hesapla" gibi fiiller ise davranıştır.

8.6 Roller

Roller de aslında ilişkilerdir ve sınıf olarak modellenmez. Öğrenci örneğine dönelim. `Student` içinde kursları `myCourses` listesinde tutuyorduk. Senaryoda "my courses" gibi bir ifade geçiyorsa bu bir roldür. Listeyi oluşturan kavramı zaten modelliyoruz: ders. Rol için ayrı sınıf açmayız.

8.7 Yazılım Sınıfları

Bir sistemin adaptörü, veri dönüşümünde kullanılan bir kütüphanenin sınıfı, grafik arayüzdeki bir butonun listesi yazılım sınıflarıdır. Bunlar tasarımı ilgilendirir. Problemi modellediğimiz için burada göstermeyiz ve eleriz.

9. Bir Aday Gerçekten Sınıf mı?

Bir kavramı işaretlediniz. Elemeli misiniz, başka bir şeyin içine özellik olarak mı eklemelisiniz, göz ardı mı etmelisiniz? Hoca bunun için şu soruları önerdi.

- **Bu kavramla ilgili veri saklayacak mıyım?** Barkod numarasına bakın. Yalnızca bir tam sayı ya da string'dir. Onunla ilgili saklanacak başka bir şey yok, başka bir şey de yapmıyor. O hâlde sınıf olmamalıdır.
- **Değişik değerler alan farklı özellikler taşıyor mu?** Bu soru, kavramın kendi özellikleri olan bir sınıf mı, yoksa yalnızca bir tanım, bir parametre mi olduğunu ortaya çıkarır.
- **Kavramdan birçok nesne türeyebilir mi?** Örneğin A müşterisi, B müşterisi, C müşterisi olacak mı?
- **Uygulamanın kapsama alanında mı?** Senaryoyu ilgilendirmiyorsa muhasebe sisteminin ayrıntısından bahsetmemek gerekir.
- **Sınırları iyi çizilmiş mi?** Bu kavramla ilgili net bir tanım yapabiliyor muyum?

Bunları olumlu cevaplayabiliyorsam o benim için bir kavramsal sınıftır. Öğrenci örneğinde durum budur: Öğrenci hakkında bilgi tutmam gerekir, birden fazla örneğini oluşturabilirim, farklı değerler alan öz nitelikleri vardır, kendine has ilişkileri ve ileride yapması gereken davranışları vardır. O hâlde problem uzayımda yer almalıdır.

10. Betimleme (Tanımlama) Sınıfı: Gerçek Dünyadan Bilinçli Uzaklaşma

Şimdiye kadar gerçek dünyayı yazılım uzayına birebir yansıtmaya çalıştık. Bazen bundan biraz uzaklaşmak gerekir. Bu, deneyimle ortaya çıkan bir şeydir ve düşünüp sorgulamayı ister.

10.1 İlk Model

Senaryolardan bir `Item` kavramı çıkardım. Ürünün tanımı, fiyatı, belki seri numarası ve barkod numarası (`itemID`) var. Bunlar ayrı sınıf değil, ürünün özellikleri. Bu model yanlış mı? Hayır. Problemi oluşturan ürün kavramını özellikleriyle birlikte paketledi. İsmi ve özellikleri var. Şu an için yeterlidir.

10.2 Sorun: Satılınca Kaybolan Bilgi

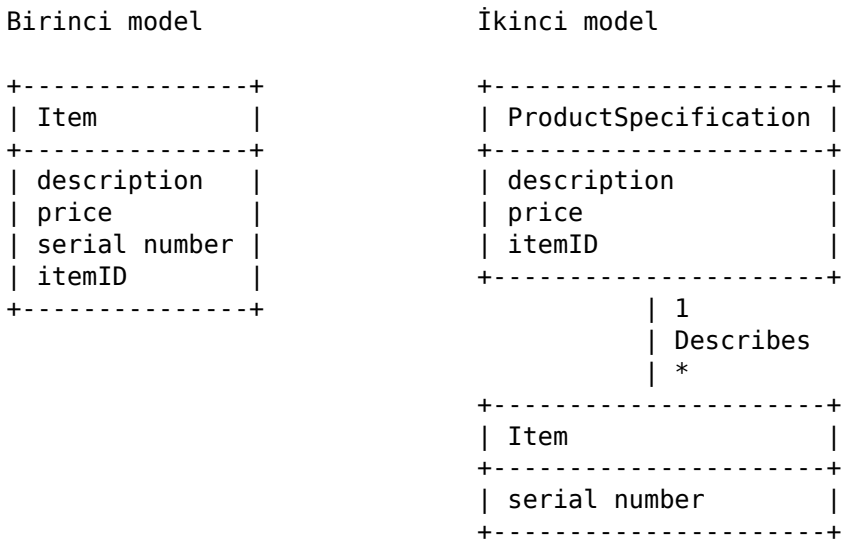
Sınıf olma sorularından biri, birden fazla örnek türetip türetemediğimizdi. Evet türetebiliyoruz: 10 çikolatalı gofret, 3 sakız. Gofreti, sakızı, kalemi, silgiyi bu sınıftan türeyen nesneler olarak kullanabiliyorum. Şimdi hocanın düşünce deneyine bakalım.

- **Verilen:** Elimde 100 gofret nesnesi var.
- **Birinci satış:** Biri 50 tane alıyor. $100 - 50 = 50$ (ara adım eklendi).
- **İkinci satış:** Biri 30 tane alıyor. $50 - 30 = 20$.
- **Üçüncü satış:** Kalan 20 tane de satılıyor. $20 - 20 = 0$.
- **Sonuç:** Elimde hiç gofret nesnesi kalmadı.

Gofret artık modelimde yer almıyor, en fazla veri tabanında "şu kadar satılan şu ürün" diye bir kayıt olarak kaldı. Ama ürünle ilgili daha sonra bilmem gereken pek çok şeyi kaybettim: tanımını, fiyatını, kod numarasını. Tasarıma geçerken bunu şöyle fark edersiniz: "Bütün item'ları kullandım, yok ettim, elimde hiç kalmadı. Bu ürünün barkod numarasını ve fiyatını bana kim verecek?"

10.3 Çözüm: Betimleme Sınıfı

Bu bilgileri ayrı bir tanımlama sınıfıyla tutarım. Buna **betimleme sınıfı (description class, specification class)** denir.



Şimdi elimdeki 50 gofreti satsam 50 `Item` nesnesi listemden gider. Ama gofret tanımı durduğu için üreticisini, kalorisini, fiyatını ve barkod numarasını tutmaya devam ederim.

Her ürüne göre değişen bilgi ise `Item` üzerinde kalır. Örneğin son kullanma tarihini ürün tanımına yazmam, `Item` sınıfına yazarım. Böylece aynı tanıma sahip farklı nesneler farklı tarihlere sahip olabilir: Bir parti gofretin son kullanma tarihi Şubat 2023, sonradan stoğa giren 100 gofretin ki Mayıs 2023 olabilir.

10.4 Neyi Kazandık, Neden Yaptık?

Birinci model gerçek hayata daha yakındı, ama yazılımda sorun çıkaracağını öngördüm. Gerçekte olmayan bir "ürün tanımı" kavramı çıkardım. Tanım sınıfı tek başına bir gerçek hayat nesnesi değildir. İki sınıf birlikte, aralarındaki ilişkiyle, gerçek hayattaki ürün kavramını oluşturur. Yani biraz gerçek hayattan uzaklaştım.

> **Hocanın vurgusu:** Birinci model yanlış değildi. İlk iterasyonda böyle bir analiz modeli yapsanız kimse yanlış yaptınız demez. İkinci modele geçmek ise daha iyi bir tasarıma yönlendirir. Bu, "hadi böyle yapalım" denecek bir şey değildir. Düşünce, sorgulama ve deneyimle iteratif olarak ortaya çıkan bir sonuçtur.

11. Bağlantıların Belirlenmesi

Kavramları, yani kutuları ve kutuların içindeki özellikleri belirledik. Bunlar iş yapan birimler değil, problemde yer alan kavramların yansımasıdır. Oysa bir sorumluluğun yerine getirilmesi için senaryodaki eylemleri de birilerine aktarmam gerekir. Kavramlar iş birliği yapacaksa aralarına bağlantılar çizmem gerekir.

11.1 Fiillerden Bağlantıya

"Kasiyer bir satış başlatır" cümlesinde, satış ile kasiyer arasında bir başlatma bağlantısı olduğunu görürüm ve çizerim. Demek ki senaryolardaki isimler değil, **fiiller ve eylemler** üzerinden yapılan analiz, kavramlar arasındaki bağlantıları verir.

11.2 Çoğullamalar

Bağlantının uçlarındaki sayılar, o ilişkide her kavramdan kaç tane yer aldığını gösterir.

Gösterim	Anlamı
`1`	Mutlaka bir tane, daha az ya da daha fazla olamaz
`0..1`	Hiç ya da en fazla bir tane
`1..*`	En az bir tane, üst sınır açık
*	Sıfır ya da n tane

Gösterim	Anlamı
`1..3`	En az bir, en çok üç tane

Örnek: İçinde hiç ürün olmayan bir satış olabilir mi? Olmaz, mantıksızdır. Satış yapılması için en az bir ürün gerekir. O hâlde satış ile ürün arasındaki ilişki en az bire n olmalıdır, sıfıra n kabul edilmez.

11.3 Bağlantı Adı ve Okuma Yönü

Bağlantılara isim veririz: “Kasiyer başlatır satış”, “öğrenci alır ders”. Okuma yönü de önemlidir. Tersten okunursa “ders alır öğrenci” olur ve sanki ders gidip öğrenciyi alıyormuş gibi bir anlam çıkabilir. Bu yüzden ismin üzerine bir ok çizip okuma yönünü belirtiriz. Yanlış okuma çok anlamsızsa yazmaya gerek olmayabilir, ama “almak” gibi genel fiillerde yönüyle göstermek daha doğrudur.

11.4 Her Fiil Bir Bağlantı Değildir

Bütün fiilleri bulamayabilirsiniz, bu çok normaldir. Önemli olan kavramları doğru bulmaktır.

> **Hocanın vurgusu:** Bütün fiilleri ve fiilimsileri işaretleyip hepsine bağlantı derseniz, bütün kavramların çapraz şekilde birbirine bağlı olduğu bir diyagram çizersiniz. Bir süre sonra modelin okunurluğunu ve anlaşılırlığını kaybedersiniz.

11.5 Bağlantılar Nasıl Bulunur?

Kavramlarda olduğu gibi iki yol vardır.

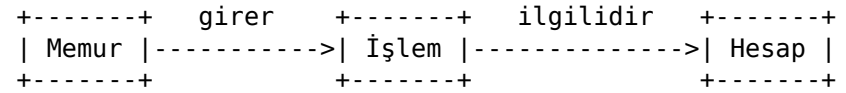
- **Yaygın bağlantı listesi:** Fiziksel barındırma, mantıksal içinde bulundurma, kayıt etme, kullanma, tanımlama, sahibi olma gibi bilinen genel ilişkiler.
- **Use case’lerdeki fiiller:** Fiillerden yola çıkarak kavramlar arasına çizgi çekmek.

11.6 Elenecek Bağlantılar

- **Elenmiş sınıflara giden bağlantılar:** O sınıflar modelde yoktur.
- **Amaç dışı bağlantılar:** Çözümün o iterasyonu ya da senaryosuyla ilgisiz olanlar.
- **Gerçekleştirmeye ait bağlantılar:** Yazılımı ilgilendirenler.
- **Faaliyetler:** “ATM kredi kartını kabul eder” gibi. ATM’nin işlem başlatmak için kartı alması, ardından kimlik doğrulama mekanizmasının çalışması birer etkileşimdir. Bunları tasarımda göstereceğiz.

11.7 Üçlü Bağlantılardan Kaçınmak

Dil yapısı nedeniyle senaryolarda üçlü bağlantılar çıkabilir. Bunlar modelde sevilen gösterimler değildir. Hocanın örneği şu cümledir: “Memur hesapla ilgili işlemleri girer.” Burada memur, hesap ve işlem vardır. Memur hesapla ilişkili, işlem de ikisiyle ilişkili görünür. Bu gösterim kafa karıştırır. Bunu ikili bağlantılara böleriz:



Hoca, “işlem” diye modellemenin de çok doğru olmadığını, örnek olsun diye böyle aldığını belirtti. Her bağlantıda iki eleman olacak şekilde çalışmak gerekir.

> **Ek bilgi (derste anlatılmadı):** UML’de ikiden fazla sınıfı bağlayan bağlantılar (n-ary association) da tanımlıdır ve bir baklava sembolüyle gösterilir. Yani üçlü gösterim dilin dışında değildir. Hocanın önerisi, modelin anlaşılır kalması için ikili bağlantıları tercih etmektir.

12. Bağlantı Örnekleri: Mağaza ve Havayolu

Çıkarmamız gereken, problemin çözümüne fayda sağlayan ve tasarımı ilgilendiren bağlantılardır. "Tip, fiil, tip" adlandırmasını doğru bir isimle yapmak da önemlidir.

12.1 Mağaza Örneği

Senaryolardan dört kavram bulunmuş: `Store` (dükkan), `Register` (kasa), `Sale` (satış), `Payment` (ödeme).

Bağlantı	Çoğullama	Hocanın okuması
Store — Contains — Register	1 — `1..*`	Sistemde tek dükkan var, içinde en az bir, en çok n kasa olabilir
Register — Sale	1 — `1..*`	Bir kasa en az bir satış gerçekleştirir. Sıfır satışlı kasanın varlığı anlamsız sayılmış
Sale — Paid-by — Payment	1 — 1	Bir satış tek bir ödemeyle ödenir

Okuma yönü: "Register contains Store" zaten mantıksızdır, "Store contains Register" doğrudur. İstenirse yön okla da belirtilir.

Tasarıma ipucu: `Store` içinde `registerList` diye bir yapı oluşturacağımı ve n tane kasayı buna bağlayacağımı daha şimdiden anlıyorum. Kasaların sahibi doğrudan `Store` olur. Bunun liste olduğu, bir stereotiple gösterilebilir. Küme ya da vektör de olabilir, ne kullandığınıza bağlıdır.

Soru: Markette 10 lirasını kartla, kalan 5 lirasını nakit ödeyebiliyorum. Çoğullama neden 1?
Cevap: Hocanın kendi sorup cevapladığı bu soruda cevap şudur: Evet böyle olabilir. Ama bu sistemin iş analizi, her satışın yalnızca tek ödemeyle yapıldığını ortaya çıkarmış. Çoğullama analize göre yazılır. Parçalı ödeme olsaydı oraya yıldız konurdu. Analiz en fazla üç ödemeye izin verseydi `1..3` yazılırdı. Bir satış nesnesi de en fazla üç ödeme nesnesiyle ilişkili olabilirdi.

12.2 Havayolu Örneği

Bir havayolunun personel ve uçak atamalarını yöneten bilgi sistemi düşünelim.

Bağlantı	Çoğullama	Hocanın okuması
Airline — Employs — Person	1 — `1..*`	Havayolunda bir ile n arası çalışan var
Person — Supervises — Person	1 — *	Bir kişi n kişiyi yönetir
Person — Assigned-to — Flight	`1..*` — *	En az bir, en çok n kişi, n uçuşa atanabilir
Plane — Assigned-to — Flight	1 — *	Bir uçak sıfır ya da n uçuşa atanabilir

Kendine dönen bağlantı: Çalışanların bir kısmı aynı zamanda müdürdür. Rollerin ayrı kavram olmadığını söylemiştik. Bu yüzden `Person` kavramından çıkıp yine ona giren bir bağlantı çizilir. Tasarımda `Person` sınıfına `supervisedBy` gibi, tipi `Person` olan bir alan eklemek yeter. Her çalışan, yöneticisini başka bir nesneye referans olarak tutar. Kavramsal modelde böyle göstermek basitlik sağlar. Teknik olmayan biri bile bakınca "Bir yönetici n kişiyi yönetiyor, bunların hepsi insan" diyebilir.

Okuma yönü: "Flight assigned to plane" okuması yanlış anlaşılabileceği için yön özellikle belirtilmiştir. "One plane is assigned to n flights" diye okunur.

Böylece problem uzayındaki kavramları aday yazılım sınıfları olarak modelde bulundurmuş oluruz. Yapılan bütün çaba budur.

13. Market Satış Sisteminin Kavramsal Sınıf Diyagramı

Süreç toparlanırsa şöyledir:

1. Bütün senaryoların üzerinden geçer, isimleri işaretler ve elersiniz. Özellikleriyle birlikte bir kavram havuzu çıkar.
2. Fiillerin üzerinden geçer, bağlantıları bulursunuz.
3. Görebildiğiniz rol tiplerini ve çoğullamaları yazarsınız.

Bunu bütün senaryolar için yaptığınızda, sistemin bütün sorumlulukları için gereken toplam kavram kümesini ve kavramların bütün bağlantılarını elde edersiniz. Sonuçta bir kavramsal sınıf diyagramı çıkar.

Bunun kavramsal olduğu şuradan anlaşılır: Yalnızca gerçek dünyadan alınmış ya da ona olabildiğince yakın kavramlar vardır, yazılım ayrıntısı yoktur ve senaryoyu ilgilendirmeyen içerik bulunmaz. Diyagram sistemin tamamını ya da yalnızca bir alt parçasını gösterebilir.

Bağlantı	Çoğullama	Hocanın anlatımı
Store — Houses — Register	1 — `1..*`	Dükkanda n kasa var
Store — Stocks — Item	1 — *	Dükkan n ürünü stoklar
ProductCatalog — Used-by — Store	1 — *	Katalog n dükkanda ayrı ayrı barındırılabilir
ProductCatalog — Contains — ProductSpecification	1 — `1..*`	Ürün tanımlarının hepsi katalogda durur
ProductSpecification — Describes — Item	1 — *	Betimleme sınıfı ürünleri tanımlar
Register — Captured-on — Sale	1 — `0..1`	Kasalar satışları kaydeder
Cashier — Works-on — Register	1 — 1	Kasada kasiyer çalışır
Sale — Contained-in — SalesLineItem	1 — `1..*`	Her satışta en az bir, en çok n satış kalemi var
Sale — Paid-by — Payment	1 — 1	Satış için bir ödeme var
Sale — Is-for — Customer	1 — 1	Her satışta bir müşteri var
Ledger — Records-accounts-for — Store	1 — 1	Dükkanın kayıtlarını tutar, ilişki birebirdir
Ledger — Logs-completed — Sale	1 — *	Altyazıda ayrıca okunmuyor
SalesLineItem — Records-sale-of — Item	`0..1` — `1..*`	Bkz. aşağıdaki not

Hoca `Ledger` için “muhasebeci, kayıt şeyi” dedi. Katalog ayrı bir sınıf olduğu için **yeniden kullanılabilirlik** de artar: İleride yazılım sınıfı olacak bu kavram, başka dükkanlarda ve başka sistemlerde alınıp kullanılabilir.

Diyagram şu senaryoyu modeller: Müşteri ürünleri alıp kasaya gelir, satış başlatılır, ürün okutulur, miktar girilir, sistem toplam tutarı ve ürün bilgisini gösterir, satış tamamlanınca ödeme yapılır, satış bilgileri muhasebe sistemine kaydedilir. Bu kavramların hepsi aralarındaki ilişkilerle birlikte analizin sonucunu resmeder.

> **Ders sonrası not:** Hoca satış kalemi ile ürün arasındaki ilişkiyi okurken “tanım” ve “item tanımı” dedi. Aynı dersteki ayrıma göre tanım `ProductSpecification`, fiziksel ürün `Item`’dır. Kısa nottaki slayt okumasına göre bu bağlantı `Item`’a gider. O hâlde okuma büyük olasılıkla şöyledir: Bir satış kalemi bir ya da n ürünü kaydeder, her ürün en çok bir satış kaleminde yer alır.

Ders sonrası not: `Register` ile `Sale` arasındaki çoğullama iki diyagramda farklıdır. Örnek diyagramda `1..*` vardı ve hoca bunu "satış yapmayan kasa anlamsız" diye açıkladı. Kısa nottaki slayt okumasına göre son diyagramda `0..1` yazar. Bu büyük olasılıkla kasanın o anda işlediği tek satışı gösterir. İkisi farklı analiz kararıdır, çoğullama senaryoya göre okunmalıdır.

Ek bilgi (derste anlatılmadı): Kısa nota göre başka kaynaklarda bu ilişki büyük olasılıkla `SalesLineItem * — 1 ProductSpecification` olarak verilir. Bu gösterimde kaç adet satıldığı büyük olasılıkla satış kaleminin bir `quantity` öz niteliğinde tutulur. Başka bir kaynakta bu çizimi görürseniz, bu dersin diyagramıyla aynı problemin farklı bir modeli olarak okuyun.

14. Öz Niteliklerin Belirlenmesi

Son diyagramda bir eksik vardır: Bütün kutuların alt tarafı boştur. Her kavramın özelliklerini de içine yazmamız gerekir.

14.1 Tanım

Özellik, nitelik ya da öz nitelik (attribute), bir nesne yaratıldığında **o nesneye özgü değer alabilen verilerin** hepsine verilen addır. Bunlar yine senaryolardaki isimlerden doğar. Dikkat edilecek bir nokta, basit veri tipleriyle ifade edilmeleridir.

14.2 Ne Zaman Ayrı Sınıf Olmalı?

Birimi olan bir değer aslında bileşik bir yapıdır ve ayrı bir kavram olarak modellemek daha doğru olabilir. Hocanın örneği fiyatın tipi olan paradır. Paranın birimi ve belli bir kur karşılığı vardır, kendine has özellikleri ve davranışları olan bir kavramdır.

Bir öz nitelik adayı şu durumlarda aslında başka bir sınıftır:

- Birden fazla alandan oluşuyorsa.
- Üzerinde işlem yapılıyorsa. Örneğin kredi kartının numarası doğru mu, kart geçerli mi diye doğrulama yapılması gerekiyorsa.
- Kendine ait nitelikleri varsa.

14.3 Kasiyer ve Kasa Örneği

Kasiyer kavramını bulmuşum. Adı var, bir de şu anda çalıştığı mevcut kasayı öz nitelik olarak eklemişim. Oysa kasa, birden fazla özelliği ve kendine has davranışları olan bir yapıdır. Bu yüzden böyle yazmak doğru değildir. Kasa, ayrı bir kavram olarak modellenmeli ve iki kavram arasına bağlantı çizilmelidir. Hoca, ilk gösterimin ilk iterasyon için kabul edilebilir olduğunu, ama basit bir özellik olmadığı fark edilince ayırmak gerektiğini söyledi. Model böylece daha anlaşılır olur.

14.4 Üçüncü Bölme: Davranışlar

Kutularda henüz üçüncü bölme yoktur. Bu bölme tasarım aşamasında eklenir ve metotları yani davranışları içerir. Bir davranış, ilişkide olduğu sınıflardan türeyen nesnelerin davranışlarını çağırır. Her birinin içinde bir algoritma, bir akış vardır. Nesneler birbirinin metotlarını çağırarak birlikte çalışır ve bir senaryo işletilmiş olur. Bunun nasıl yapıldığı tasarım konusunda işlenecektir.

14.5 Öz Nitelik Ayrıntılarının Gösterimi

Öz niteliklere şu ayrıntılar eklenebilir.

Gösterim	Anlamı	Hocanın açıklaması
`-`	private	Dışarıya açmıyorum, başkalarının okumasına izin vermiyorum
`+`	public	Herkese açtığım özellik
`+ pi: Real = 3.14 {readOnly}`	public, tipli, ilk değerli, salt okunur	`Math` sınıfındaki pi herkesçe okunabilir, kimse üzerine yazamaz, 3.14 sabittir
`/`	türetilmiş (derived)	Kendi başına bilinmez, başka niteliklerle hesaplanır. Değeri bulmak için bir metot çalışır
`middleName [0..1]`	çoğullama	İkinci isim yok ya da en fazla bir tane olabilir

> **Ders sonrası not:** Hoca 01:14:10 civarında "bir bölü görüyorsanız bu bir private özellik" dedi. Doğrusu şudur: UML'de `/` yalnızca değerin türetilmiş olduğunu gösterir, görünürlükle ilgisi yoktur. Görünürlük `+` (public), `-` (private), `#` (protected) ve `~` (package) ile gösterilir. Türetilmiş bir öz nitelik public de olabilir.
>
> **Ders sonrası not:** Hoca `readOnly` için "stereotip" dedi. Doğrusu şudur: UML'de süslü parantezle yazılan `{readOnly}` bir özellik belirtecidir. Stereotipler «`list`» gibi açılı tırnakla yazılır.

Çoğullama örneğinde kişinin ilk adı ve soyadı zorunlu, ikinci adı isteğe bağlıdır. Hoca, İspanyollarda çok sayıda isim olabileceğini söyledi. Kaç orta isme izin veriyorsanız üst sınırı ona göre yazarsınız. Türkiye'de de üç isimli pek çok kişi vardır.

14.6 Bu Ayrıntılar Neden Eklenir?

Soru: Yalnızca kavramları, ilişkileri ve özellikleri göstermeye çalışmıyor muyduk? Public mi, private mi, derived mi olduğu bizi neden ilgilendirsin? **Cevap:** Hocanın kendi sorup cevapladığı bu soruda cevap şudur: Bunlar bu aşama için gerçekten fazla ayrıntıdır. Ancak tasarıma özellikle taşımak istediğiniz noktalarda "buna dikkat edin, bunun böyle olması gerekiyor" demek için eklenir. Açıklama kutusu bile eklenebilir. Bunları ifade etmenin daha formal yolları ve UML'in yetersiz kaldığı yerlerde kullanılan ilave araçlar vardır, ama bu ders kapsamında onlara girilmeyecektir.

15. Modelin Son Hâli ve Modeli Okuyanlar

Kavramları, ilişkileri, özellikleri ve çoğullamaları belirledik. Görebildiğimiz kadarıyla tipleri, görünürlükleri, okuma yönlerini ve rolleri de yazdık. Son diyagram öncekinin aynısıdır, ama kutular doludur:

- `ProductSpecification`: `itemID`, `description`, `price`
- `Item`: `serialNumber`
- `Store`: `name`, `address`
- `Register`: `ID`
- `Cashier`: `ID`, `name: String`

Hoca derste eksik kalan `serialNumber` ve kasiyerin `name` bilgisini ekranda ekledi. Tipi yazmak (`name: String`) zorunlu değildir, ama tasarıma yol göstermek istiyorsanız yazabilirsiniz. Diyagramdaki bir öz nitelik türetilmiştir, hesap yapmayı gerektirir. Bu diyagramda artı ve eksi işaretleri yoktur.

Aynı diyagramı farklı kişiler farklı amaçla okur:

- **Analist:** Senaryoları, kavramları, kavramlar arasındaki ilişkileri, öz nitelikleri, çoğullamaları ve rolleri görür. "Bu benim analiz modelimi ifade ediyor" der.
- **Tasarımcı ya da yazılımcı:** "Bunu listede tutarız, bu türetilmiş, ona bir metot yazacağım" diye düşünür. "Onun bilgileri nereden gelecek, bununla ilgili senaryo var mı?" diye sorar. Eksik varsa analiz modelini güncelletir.

- **Müşteri:** Analist çizimi önüne koyup biraz anlatır. Müşteri "İyi, ama benim aslında iki katalogum var, hem hazır ürün satıyorum hem üretip satıyorum" diyebilir. O zaman modelin o kısmı güncellenir, senaryolar etkileniyorsa onlar da elden geçirilir.

16. Analizin Sonu, Tasarıma Geçiş ve SRS Dokümanı

16.1 Şimdiye Kadar Ne Yaptık?

1. **İş analizi:** Business requirement dokümanı oluşturduk. Nesneye yönelik çözümleme yapacaksak use case'leri yazdık. Senaryoları aktörleri, bağlantıları ve ilişki türleriyle birlikte use case diyagramında gösterdik.
2. **Çözümleme:** Gereksinimlerden kavramsal bir çözüm önerisi çıkardık. Geçen hafta bunu veri akış diyagramlarıyla işlevsel olarak yaptık. Bu hafta kavramları, birlikte nasıl bulunduklarını, ilişkilerini ve özelliklerini ortaya koyduk.

Hocanın gösterdiği slaytta ortada use case modeli vardır. Bu, analizin birinci adımıdır ve sorumluluklar buradan çıkar. Slaytta bulunan system sequence diagram'ı hoca göz ardı edebileceğimizi söyledi. Senaryolardan **domain model** oluşturulur. Domain nesneleri doğrudan senaryolardan beslenir. Bu da analizin ikinci adımıdır, yani problem uzayının modellenmesi ya da kavramsal çözümleme.

16.2 Tasarıma Bir Bakış

Önümüzdeki haftalarda kavramsal sınıflardan bir tasarım modeli oluşturacağız. Kasa, katalog ve satış sınıflarından türeyen nesneler birlikte çalışacak. Hoca ürün girme işini örnek verdi:

1. Kod numarası barkoddan gelir, miktar klavyeden okunur. Bu iki parametreyle kasanın `enterItem` metodu çağrılır.
2. Kasa, kataloğa "Şu kod numaralı ürünün bilgilerini ver" der. Ürün bilgileri nesne hâlinde geri gelir ve ekranda gösterilir. Metodun içinde başka işler de yapılır, bunlar algoritmanın parçasıdır.
3. Kasa, satış nesnesine "Şu adette, şu özelliklere sahip ürünü bir satış satırı olarak oluştur ve kendi içinde tut" der. Hoca bu çağrıyı "add line item" diye okudu.

   Ders sonrası not: Hoca katalogdan dönen bilgileri sayarken "seri numarası" da dedi. Bu dersin modeline göre seri numarası `Item` üzerindedir. Katalogdaki `ProductSpecification` kod numarası, açıklama ve fiyatı tutar.

Sistem çok büyükse birden fazla kavramsal sınıf diyagramı oluşturabilirsiniz. Çok karmaşık değilse tek bir büyük diyagramda bütün kavramları barındırabilirsiniz. Bunu çizdiğinizde analiz modeli tamamlanır.

16.3 SRS Dokümanı

Analizin sonunda **SRS (Software Requirement Specification)** dokümanı ayrıntılarıyla yazılır. İçinde çözümleme de yer alır:

- **İşlevsel çözümleme yapıldıysa:** Veri akış diyagramları, süreç tanım formları ve onların içindeki sözde kod ve karar tabloları.
- **Nesneye yönelik çözümleme yapıldıysa:** Senaryolar, senaryolardan oluşturulan domain model yani kavramsal sınıf diyagramı ve ilgili açıklamalar.

  Hocanın vurgusu: SRS dokümanı örneklerine mutlaka bakın. Süre kısıtı nedeniyle derste aktarılamadı.

Dersin Toparlaması

Bu derste nesneye yönelik çözümlemenin nasıl yapıldığını gördük. Amaç, gerçek dünyadaki problemin doğru, anlaşılır ve sınanabilir bir yazılım replikasını kurmaktır. Yol şudur: Use case'lerdeki isimlerden aday sınıflar çıkarılır, fazlalık, ilgisiz, belirsiz, nitelik, işlem, rol ve yazılım sınıfı olanlar elenir. Fiillerden bağlantılar çizilir, çoğullamalar ve okuma yönleri yazılır. Son olarak öz nitelikler eklenir ve sonuç UML kavramsal sınıf diyagramı olarak gösterilir.

Betimleme sınıfı örneğinin gösterdiği gibi model iteratif olarak gelişir ve gerektiğinde gerçek dünyadan bilinçli olarak uzaklaşılır.

Böylece analiz aşaması bitti ve çıktısı SRS dokümanına yazılır. Sonraki derslerde sistem tasarımının adımlarına geçilecek. Burada bulunan kavramlara sorumluluklar atanacak ve kutuların üçüncü bölmesi olan metotlar eklenecektir.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.